Executive Summary

In May 2019, Microsoft released an out-of-band patch update for remote code execution vulnerability CVE-2019-0708, which is also known as "BlueKeep" and resides in code to Remote Desktop Services (RDS). This vulnerability is pre-authentication and requires no user interaction, making it particularly dangerous as it has the unsettling potential to be weaponized into a destructive exploit. If successfully exploited, this vulnerability could execute arbitrary code with "system" privileges. The Microsoft Security Response Center advisory indicates this vulnerability may also be wormable, a behavior seen in attacks including Wannacry and EsteemAudit. Understanding the seriousness of this vulnerability and its potential impact to the public, Microsoft took the rare step of releasing a patch for the no longer supported Windows XP operating system, in a bid to protect Windows users.

With potential global catastrophic ramifications, Unit 42 researchers felt it was important to analyze this vulnerability to understand the inner workings of RDS and how it could be exploited. Our research dives deep into the RDP internals and how they can be leveraged to gain code execution on an unpatched host. This blog discusses how Bitmap Cache protocol data unit (PDU), Refresh Rect PDU, and RDPDR Client Name Request PDU can be used to write data into kernel memory.

Since the patch was released in May, this vulnerability has received a lot of attention from the Computer Security industry. It is only a matter of time before a working exploit is released in the wild. The findings of our research highlight the risks if vulnerable systems are left unpatched.

Bitmap Cache PDU

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Manage My Cookie Settings

Copyright © 2025 Palo Alto Networks. All Rights Reserved

List PDU Data and embeds in the Persistent Key List PDU. The Persistent Key List PDU is an RDP Connection Sequence PDU sent from client to server during the

Connection Finalization phase of the RDP Connection Sequence, as shown in Figure 1.



This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

Figure 1. Remote Desktop Protocol (RDP) connection sequence

The Persistent Key List PDU header is the general RDP PDU header and is constructed as follows and shown in Figure 2: tpktHeader (4 bytes) + x224Data (3 bytes) + mcsSDrq (variable) + securityHeader (variable).



Figure 2. Client Persistent Key List PDU

Per MS-RDPBCGR documentation, the TS_BITMAPCACHE_PERSISTENT_LIST_PDU is a structure that contains a list of cached bitmap keys saved from Cache Bitmap (Revision 2) Orders ([MS-RDPEGDI] section 2.2.2.2.1.2.3) that were sent in previous sessions as shown in Figure 3.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 3. Persistent Key List PDU Data (BITMAPCACHE PERSISTENT LIST PDU)

By design, the Bitmap Cache PDU is used for the RDP client to notify the server that it has a local copy of the bitmap associated with the key, which indicates that the server does not need to retransmit the bitmap to the client. Based on the MS-RDPBCGR documentation, the Bitmap PDU has four characteristics:

- The RDP server will allocate a kernel pool to store the cached bitmap keys.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

- The Bitmap Cache PDU can be sent legitimately multiple times because the bitmap keys can be sent in more than one Persistent Key List PDU, with each PDU being marked using flags in the bBitMask field.
- There is a limit to 169 for the number of bitmap keys.

Based on these four characteristics of BITMAPCACHE PERSISTENT LIST PDU, it appears to be a good candidate to write arbitrary data into the kernel if either the number of bitmap keys limit to 169 can be bypassed, or the RDP developers in Microsoft didn't implement it according to that limit.

How to write data into kernel with Bitmap Cache PDU

According to MS-RDPBCGR documentation, a normal decrypted BITMAPCACHE PERSISTENT LIST PDU is shown below:

```
f2 00 -> TS_SHARECONTROLHEADER::totalLength = 0x00f2 = 242 bytes

17 00 -> TS_SHARECONTROLHEADER::pduType = 0x0017

0x0017

= 0x0010 | 0x0007

= TS_PROTOCOL_VERSION | PDUTYPE_DATAPDU

ef 03 -> TS_SHARECONTROLHEADER::pduSource = 0x03ef = 1007

ea 03 01 00 -> TS_SHAREDATAHEADER::shareID = 0x000103ea

00 -> TS_SHAREDATAHEADER::pad1
```

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

PDUTYPE2_BITMAPCACHE_PERSISTENT_LIST (43)

00 -> TS_SHAREDATAHEADER::generalCompressedType = 0

00 00 -> TS_SHAREDATAHEADER::generalCompressedLength = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::numEntries[0] = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::numEntries[1] = 0

19 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::numEntries[2] = 0x19 = 25

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::numEntries[3] = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::numEntries[4] = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::totalEntries[0] = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::totalEntries[1] = 0

19 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::totalEntries[2] = 0x19 = 25

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::totalEntries[3] = 0

00 00 -> TS_BITMAPCACHE_PERSISTENT_LIST::totalEntries[4] = 0

03 -> TS_BITMAPCACHE_PERSISTENT_LIST::bBitMask = 0x03

0x03

= 0x01 | 0x02

= PERSIST_FIRST_PDU | PERSIST_LAST_PDU

00 -> TS_BITMAPCACHE_PERSISTENT_LIST::Pad2

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

48 29 22 78 -> Cache 2, Key 0, High 32-bits
(TS_BITMAPCACHE_PERSISTENT_LIST_ENTRY::Key2)

61 f7 89 9c -> Cache 2, Key 1, Low 32-bits
(TS_BITMAPCACHE_PERSISTENT_LIST_ENTRY::Key1)

cd a9 66 a8 -> Cache 2, Key 1, High 32-bits
(TS_BITMAPCACHE_PERSISTENT_LIST_ENTRY::Key2)

...

In kernel module RDPWD.sys, the function routine ShareClass::SBC_HandlePersistentCacheList is responsible for parsing BITMAPCACHE PERSISTENT LIST PDU. When the bBitMask field in the structure is set to a bit value of 0x01, it indicates the current PDU is PERSIST FIRST PDU. SBC_HandlePersistentCacheList will then call WDLIBRT_MemAlloc to allocate a kernel pool (allocate kernel memory) to store persistent bitmap cache keys as shown in Figure 4. A value of 0x00 indicates the current PDU is PERSIST MIDDLE PDU. A value of 0x02 indicates the current PDU is PERSIST LAST PDU. When parsing PERSIST MIDDLE PDU and PERSIST LAST PDU, SBC_HandlePersistentCacheList will copy bitmap cache keys to the memory allocated before as shown in Figure 5.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 4. SBC_HandlePersistentCacheList pool allocation and totalEntriesCacheLimit check

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 5. SBC_HandlePersistentCacheList copy bitmap cache keys

The stack trace on Windows 7 x86 and the second argument to TS_BITMAPCACHE_PERSISTENT_LIST structure of SBC_HandlePersistentCacheList are shown in Figure 6 and Figure 7.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 6. SBC_HandlePersistentCacheList stack trace



This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved

Figure 7. TS_BITMAPCACHE_PERSISTENT_LIST structure as the second argument of SBC_HandlePersistentCacheList

As seen in Figure 4, bitmapCacheListPoolLen = 0xC * (total length + 4) and the total length = totalEntriesCache0 + totalEntriesCache1 + totalEntriesCache2 + totalEntriesCache3 + totalEntriesCache4. Based on this formula we can set "WORD value" totalEntriesCache**X**=0xffff to make the bitmapCacheListPoolLen to the maximum value. However, there is a totalEntriesCacheLimit check for each totalEntriesCache**X** shown in Figure 8. The totalEntriesCacheLimit**X** is from the TS_BITMAPCACHE_CAPABILITYSET_REV2 structure, which is initiated in the CAPAPI_LOAD_TS_BITMAPCACHE_CAPABILITYSET_REV2 function when calling DCS_Init by RDPWD, shown in Figure 8. This will be combined in the CAPAPI_COMBINE_TS_BITMAPCACHE_CAPABILITYSET_REV2 function when parsing active confirm PDU, as shown in Figure 9.



Figure 8. RDPWD!CAPAPI_LOAD_TS_BITMAPCACHE_CAPABILITYSET_REV2

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 9. RDPWD!CAPAPI_COMBINE_TS_BITMAPCACHE_CAPABILITYSET_REV2

CAPAPI_COMBINE_TS_BITMAPCACHE_CAPABILITYSET_REV2 will combine the server initiated NumCellCaches (0x03) and totalEntriesCacheLimit[0-4] (0x258, 0x258, 0x10000, 0x0, 0x0) with client request NumCellCaches (0x03) and totalEntriesCache[0-4] (0x80000258, 0x80000258, 0x8000fffc, 0x0, 0x0), shown with edx and esi registers in Figure 9. The client can control NumCellCaches and totalEntriesCache[0-4], shown in Figure 10, but they cannot be over the server initiated NumCellCaches (0x03) and totalEntriesCacheLimit[0-4] (0x258, 0x258, 0x10000, 0x0, 0x0) shown in Figure 11.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 10. TS_BITMAPCACHE_CAPABILITYSET_REV2

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 11. CAPAPI_COMBINE_TS_BITMAPCACHE_CAPABILITYSET_REV2 function

With this knowledge we can compute the maximum bitmapCacheListPoolLen = oxC *

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 12. Persistent Key List PDU Memory dump

However, we observed that not all data can be controlled by the RDP client in bitmap cache list pool as expected. There is a 4 byte uncontrolled data (the index value) between each 8 bytes controlled data which is not friendly for shellcode. Additionally the 0xc3870 sized kernel pool cannot be allocated multiple times due to the fact the Persistent Key List PDU can only be sent once legitimately. However, there are still

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 13. Persistent Key List PDU statistical characteristics

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved

structure includes the general PDU header and the refreshRectPduData (variable) shown in Figure 14.



Figure 14. Refresh Rect PDU Data

The numberOfAreas field is an 8-bit unsigned integer to define the number of Inclusive Rectangle structures in the areasToRefresh field. The areaToRefresh field is an array of TS_RECTANGLE16 structures shown in Figure 15.



This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved

session screen area. It is based on default opened channel with the channel ID 0x03ea (Server Channel ID). After the connection sequence is finished, as shown in Figure 1, Refresh Rect PDU can be received/parsed by the RDP server and most importantly, can be sent for multiple times legitimately. Although limited to only 8 bytes for TS_RECTANGLE16 structure, which means only 8 bytes and not massive data can be controlled by the RDP client, it is still a very good candidate to write arbitrary data into the kernel.

How to write data into kernel with Refresh Rect PDU

A normal decrypted Refresh Rect PDU is shown in Figure 16.



This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

The kernel module RDPWD.sys code function WDW_InvalidateRect is responsible for parsing Refresh Rect PDU as seen in Figure 17, below.



Figure 17. RDPWD!WDW_InvalidateRect stack trace

As shown in Figure 18, WDW_InvalidateRect function will parse Refresh Rect PDU stream and retrieve the numberOfAreas field from the stream as the loop count. Being a byte type field, the maximum value of numberOfAreas is 0xFF, so the maximum loop count is 0xFF. In the loop, WDW_InvalidateRect function will get left, top, right, and bottom fields in TS_RECTANGLE16 structure, put them in a structure on the stack and make it as the 5th parameter of WDICART_IcaChannelInput. To be mentioned here, the 6th parameter of WDICART_IcaChannelInput is the constant 0x808, and we will show how it helps for an efficient spray.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 18. RDPWD!WDW_InvalidateRect function

WDICART_IcaChannelInput will eventually call kernel module termdd.sys function IcaChannelInputInternal. As shown in Figure 19, if a series of condition checks are True, the function IcaChannelInputInternal will call ExAllocatePoolWithTag to allocate an inputSize_6th_para + 0x20 sized kernel pool. As such, when the function IcaChannelInputInternal is called by RDPWD!WDW_InvalidateRect, inputSize_6th_para=0x808, and the size of the kernel pool is 0x828.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 19. termdd!IcaChannelInputInternal ExAllocatePoolWithTag and memcpy

If the kernel pool allocation is successful, memcpy will be called to copy input_buffer_2 to the newly allocated kernel pool memory. Figure 20 shows the parameters of memcpy when the caller is RDPWD!WDW_InvalidateRect.



Figure 20. termdd!IcaChannelInputInternal memcpy windbg dump

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 21. RDPWD!WDW_InvalidateRect stRect structure set

Using this information, when the RDP client sends one Refresh Rect PDU with the numberOfAreas field of 0xFF, the RDP server will call termdd!IcaChannelInputInternal 0xFF times. Each termdd!IcaChannelInputInternal call will allocate 0x828 kernel pool memory and copy eight bytes of client controlled TS_RECTANGLE16 structure to that kernel pool. So, one Refresh Rect PDU with numberOfAreas field of 0xFF will allocate 0xFF number of 0x828 sized kernel pools. In theory if the RDP client sends Refresh Rect PDU 0x200 times, the RDP server will allocate around 0x20000 of 0x828 size non-paged kernel pools. Considering 0x828 sized kernel pool will be aligned by 0x1000, they will span a very large scope of the kernel pool and at the same time, client controlled eight bytes of data would be copied at the fixed 0x02c offset in each 0x1000 kernel pool. This is demonstrated in Figure 22 we get a stable pool spray in the kernel with Refresh Rect PDU.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 22. RDPWD!WDW_InvalidateRect spray

There are situations where ExAllocatePoolWithTag and memcpy are not be called when a pointer (represented as variable v14 in Figure 23) is modified by termdd!_IcaQueueReadChannelRequest and the comparison will be False as shown in Figure 23, the route will enter routine _IcaCopyDataToUserBuffer which leads to an unsuccessful pool allocation. However, when sending Refresh Rect PDU many times, we can still get a successful kernel pool spray even though there are some unsuccessful pool allocations.

Besides, there are situations where some kernel pools may be freed after the RDP server is finished using them, but the content of the kernel pool will not be cleared, making the data which we spray into the kernel valid to use in the exploit.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 23. termdd!IcaChannelInputInternal IcaCopyDataToUserBuffer

RDPDR Client Name Request PDU

Per MS-RDPEFS documentation RDPDR Client Name Request PDU is specified in [Remote Desktop Protocol: File System Virtual Channel Extension] which runs over a static virtual channel with the name RDPDR. The purpose of the MS-RDPEFS protocol is to redirect access from the server to the client file system. Client Name Request is the second PDU sent from client to server as shown in Figure 24.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved



This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 25. Client Name Request (DR_CORE_CLIENT_NAME_REQ)

The header is four bytes RDPDR_HEADER with the Component field set to RDPDR_CTYP_CORE and the PacketId field set to PAKID_CORE_CLIENT_NAME. The ComputerNameLen field (4 bytes) is a 32-bit unsigned integer that specifies the number of bytes in the ComputerName field. The ComputerName field (variable) is a variable-length array of ASCII or Unicode characters, the format of which is determined by the UnicodeFlag field. This is a string that identifies the client computer name.

How to write data into kernel with RDPDR Client Name Request PDU

The following can be said about the RDPDR Client Name Request PDU. The Client

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

it an excellent choice to write data into the kernel memory. A typical RDPDR Client Name Request PDU is shown in Figure 26.



Figure 26. client name request memory dump

When the RDP server receives a RDPDR Client Name Request PDU, the function IcaChannelInputInternal in the kernel module termdd.sys is called to dispatch channel data first, then the RDPDR module will be called to parse the data part of the Client Name Request PDU. The function IcaChannelInputInternal for Client Name Request PDU applies the same code logic as for Refresh Rect PDU. It will call ExAllocatePoolWithTag to allocate kernel memory with tag TSic and use memcpy to copy the client name request data to the newly allocated kernel memory as shown in Figure 27.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved



Figure 27. client name request

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. **Privacy statement**

Copyright © 2025 Palo Alto Networks. All Rights Reserved

vulnerability exploit and also can be used to write the shellcode into the kernel pool, even can be used to spray consecutive client controlled data into the kernel memory.

As shown in Figure 28 we successfully obtained a stable pool allocation and write client-controlled data into the kernel pools with RDPDR Client Name Request PDU.



Figure 28. client name request stable pool allocation

Detection and Mitigation

CVE-2019-0708 is a severe vulnerability targeting RDP and can be exploitable with unauthenticated access. According to the MSRC advisory, Windows XP, Windows 2003, Windows 7 and Windows 2008 are all vulnerable. Organizations using those Windows

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved

- Traps prevents exploitation of this vulnerability on Windows XP, Windows 7, and Windows Server 2003 and 2008 hosts.
- Threat Prevention detects the scanner/exploit.

Conclusion

In this blog we introduced three ways to write data into the kernel with RDP PDU.

- Bitmap cache PDU allows the RDP server to allocate a 0xc3870 sized kernel pool after a 0x2b5200 sized pool allocation and write controllable data into it, but cannot perform the 0xc3870 sized kernel pool allocation multiple times.
- Refresh Rect PDU can spray many 0x828 sized kernel pools which are 0x1000 aligned and write 8 controllable bytes into each 0x828 sized kernel pool.
- RDPDR Client Name Request PDU can spray controllable sized kernel pool and fill them with controllable data.

We believe that there are other yet-to-be-documented ways to make CVE-2019-0708 exploitation easier and more stable. Users should take steps to ensure their vulnerable systems are protected through one of the mitigation steps listed above.

Thank you to Mike Harbison for his assistance in editing this report.

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. By continuing to browse this site, you acknowledge the use of cookies. Privacy statement

Copyright © 2025 Palo Alto Networks. All Rights Reserved